Чисельні методи

Лабораторна робота №4

Інтерполювання функцій

Виконав:

Студент КН-24

Бевзюк Н.С.

2026

Варіант **1**

1. Мета роботи.

Метою лабораторної роботи є здобуття практичних навичок розроблення алгоритмів і програм для глобального та локального інтерполювання функцій поліномами Лагранжа. У роботі досліджується апроксимація заданої функції за допомогою трьох підходів: локальної лінійної, локальної квадратичної та глобальної інтерполяції.

Додатково мета полягає в тому, щоб порівняти точність різних видів інтерполювання на довільно вибраній точці всередині інтерполяційного інтервалу, а також наочно побачити відмінність між точною функцією та інтерполяційними кривими на спільному графіку.

2. Завдання.

1) Побудувати графік заданої функції $y = 4 - \cos(x^2)$ та вибрати на ньому інтерполяційний інтервал.

2) Обрати шість вузлів інтерполяції у вигляді точок x на цьому інтервалі, обчисливши для них значення функції.

3) Для довільної точки x всередині інтерполяційного інтервалу, яка не збігається з вузлами, обчислити значення функції в цій точці та значення, отримані трьома видами інтерполяції: локальною лінійною, локальною квадратичною і глобальною.

4) Для кожного методу знайти відносну похибку за формулою

$$\delta = \left| \frac{y_{\text{таб}} - y_{\text{точн}}}{y_{\text{точн}}} \right| \cdot 100\%.$$

5) Побудувати спільні графіки точної функції та трьох інтерполяційних наближень для подальшого порівняння.

3. Розрахункові формули чисельного методу.

Метою даної роботи є наближення значень функції за допомогою інтерполяційних поліномів. Для розв’язання задачі використовуються наступні математичні підходи.

Глобальна інтерполяція передбачає побудову єдиного аналітичного виразу (полінома), який проходить через усі задані вузли інтерполяції $(x_0, y_0), (x_1, y_1), \ldots, (x_n, y_n)$. Для n+1 вузлів будується поліном ступеня n.

Загальний вигляд інтерполяційного полінома Лагранжа:

$L_n(x) = \Sigma_{i=o}^{n} y_i \cdot l_i(x)$

де $l_i(x)$ — базисні поліноми (коефіцієнти Лагранжа), що визначаються за формулою:

$l_i(x) = \prod_\{j=0, j\neq i\}^\{n\} (x - x_j) / (x_i - x_j)$

Даний метод забезпечує високу точність на всьому інтервалі, проте при великій кількості точок може виникати осциляція (ефект Рунге).

При локальній інтерполяції для кожного підінтервалу $[x_i, x_{i+1}]$ будується окрема функція. Лінійна інтерполяція з'єднує сусідні вузли відрізками прямих.

Значення функції в точці x, що належить відрізку $[x_i, x_{i+1}]$, обчислюється за формулою: $f(x) \approx y_i + ((y_{i+1} - y_i) / (x_{i+1} - x_i)) \cdot (x - x_i)$

Цей метод є найпростішим, але має нижчу точність, оскільки графік у вузлах має злами (неперервна лише сама функція, але не її похідна).

Квадратична інтерполяція використовує параболічне наближення на основі трьох найближчих до точки x вузлів. Це забезпечує гладкість наближення вищого порядку порівняно з лінійною інтерполяцією.

Для трьох вузлів
(x_{i-1}, y_{i-1}), (x_i, y_i) та (x_{i+1}, y_{i+1}) розрахункова формула має вигляд:

$$P_2(x) = y_{i-1} \cdot ((x - x_i)(x - x_{i+1}) / ((x_{i-1} - x_i)(x_{i-1} - x_{i+1})))$$
$$+ y_i \cdot ((x - x_{i-1})(x - x_{i+1}) / ((x_i - x_{i-1})(x_i - x_{i+1})))$$
$$+ y_{i+1} \cdot ((x - x_{i-1})(x - x_i) / ((x_{i+1} - x_{i-1})(x_{i+1} - x_i)))$$

Для оцінки точності побудованих моделей у точці x, яка не збігається з вузлами інтерполяції, розраховується відносна похибка δ (у відсотках):

$$\delta = \left| \frac{y_{\text{таб}} - y_{\text{точн}}}{y_{\text{точн}}} \right| \cdot 100\%.$$

4. Обрані вузли інтерполяції.

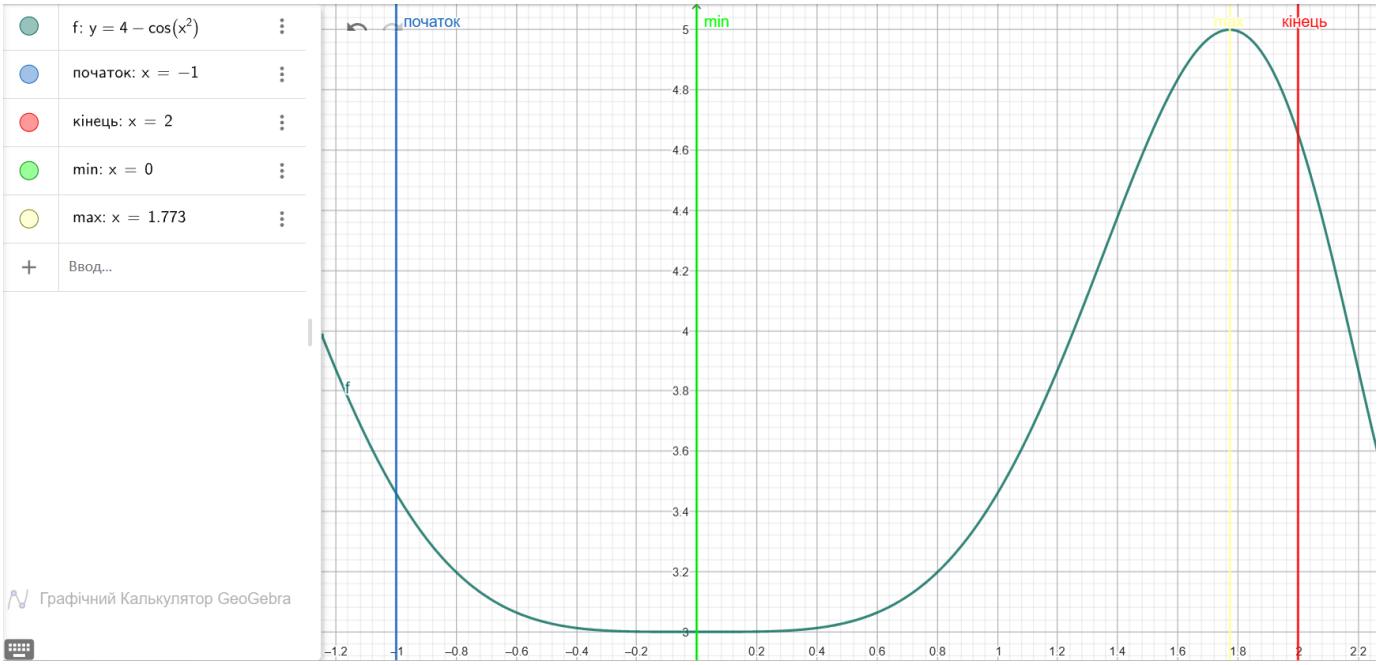
На першому етапі за допомогою математичного пакета GeoGebra було побудовано графік функції $y=4-\cos(x^2)$. Це дозволило наочно визначити поведінку функції та зафіксувати деякі необхідні точки, які стануть вузлами інтерполяції:

Початок відрізку -1

Кінець відрізку 2

Максимум на відрізку 1,773

Мінімум на відрізку 0



Малюнок 1. Графік функції $y=4-\cos(x^2)$

Окрему увагу було приділено розрахунку точки перегину, де змінюється напрям опуклості графіка. Для цього знайдено другу похідну функції та прирівнянємо її до нуля:

$y' = 2x \sin(x^2)$

$y'' = 2\sin(x^2) + 4x^2 \cos(x^2)$

Розв'язання рівняння $2\sin(x^2) + 4x^2 \cos(x^2)= 0$ на заданому інтервалі дає корінь $x=1{,}355$.

Для забезпечення рівномірності вузлів була обрана довільна точка $x = 0{,}8$.

Отже, остаточний набір із шести вузлів інтерполяції, впорядкованих за зростанням: **-1; 0; 0,8; 1,355; 1,773; 2**

5. Текст програми.

ПОЧАТОК КОДУ

```
import math
import tkinter as tk
from tkinter import ttk, messagebox
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg,
NavigationToolbar2Tk

# y = 4 - cos(x^2)
def f_exact(x: float) -> float:
```

```
    return 4.0 - math.cos(x ** 2)

# Загал ф полінома Лагранжа
def lagrange_value(xs, ys, xq):
    total = 0.0
    n = len(xs)

    for i in range(n):
        term = ys[i]
        for j in range(n):
            if i != j:
                term *= (xq - xs[j]) / (xs[i] - xs[j])
        total += term
    return total

# вибір вузлів для локальної лін. інтерполяції
def select_linear_nodes(xs, xq):
    if xq <= xs[0]:
        return 0, 1
    if xq >= xs[-1]:
        return len(xs) - 2, len(xs) - 1

    idx = np.searchsorted(xs, xq) - 1
    idx = max(0, min(idx, len(xs) - 2))
    return idx, idx + 1

# вибір вузлів для локальної  квадратичної інтерполяції
def select_quadratic_nodes(xs, xq):
    order = sorted(range(len(xs)), key=lambda i: abs(xs[i] - xq))
    idxs = sorted(order[:3])
    return idxs

# локал. лін. інтерполяція для 1 точки
def local_linear_value(xs, ys, xq):
    i, j = select_linear_nodes(xs, xq)
    return lagrange_value([xs[i], xs[j]], [ys[i], ys[j]], xq)

# локал. квадратична інтерполяція для 1 точки
def local_quadratic_value(xs, ys, xq):
    idxs = select_quadratic_nodes(xs, xq)
    x_part = [xs[i] for i in idxs]
    y_part = [ys[i] for i in idxs]
    return lagrange_value(x_part, y_part, xq)
```

```
# відносна похибка
def relative_error(y_exact, y_interp):
    return abs((y_interp - y_exact) / y_exact) * 100.0

# значення для побудови графіка
def piecewise_linear_values(xs, ys, grid_x):
    values = []
    for x in grid_x:
        values.append(local_linear_value(xs, ys, x))
    return np.array(values, dtype=float)

def piecewise_quadratic_values(xs, ys, grid_x):
    values = []
    for x in grid_x:
        values.append(local_quadratic_value(xs, ys, x))
    return np.array(values, dtype=float)

# GUI
class InterpolationApp(tk.Tk):
    def __init__(self):
        super().__init__()

        self.title("Лабораторна робота №4 — Інтерполювання функцій")
        self.geometry("1100x760")
        self.minsize(1000, 700)

        self.x_nodes = []
        self.y_nodes = []
        self.x_query = None
        self.results = {}

        self._build_ui()

    # інтерфейс
    def _build_ui(self):
        top = ttk.Frame(self, padding=10)
        top.pack(fill=tk.X)

        ttk.Label(
            top,
            text="Функція: y = 4 - cos(x^2)",
            font=("Arial", 12, "bold")
```

```
        ).grid(row=0, column=0, columnspan=6, sticky="w", pady=(0, 8))

        ttk.Label(
            top,
            text="Вузли інтерполяції (вводіть числа через крапку):"
        ).grid(row=1, column=0, columnspan=6, sticky="w", pady=(0, 8))

        self.x_entries = []
        default_x = [-1.0, 0.0, 0.8, 1.355, 1.773, 2.0]

        nodes_row = ttk.Frame(top)
        nodes_row.grid(row=2, column=0, columnspan=6, sticky="w", pady=(0, 5))

        for i in range(6):
            frame = ttk.Frame(nodes_row)
            frame.pack(side=tk.LEFT, padx=8)

            ttk.Label(frame, text=f"x{i+1}:").pack(side=tk.LEFT)
            entry = ttk.Entry(frame, width=10)
            entry.pack(side=tk.LEFT, padx=4)
            entry.insert(0, str(default_x[i]))
            self.x_entries.append(entry)

        query_frame = ttk.Frame(top)
        query_frame.grid(row=3, column=0, columnspan=6, sticky="w", pady=(10, 3))

        ttk.Label(query_frame, text="Точка для обчислення x* (не повинна збігатися
з вузлами):").pack(side=tk.LEFT)
        self.query_entry = ttk.Entry(query_frame, width=12)
        self.query_entry.pack(side=tk.LEFT, padx=8)
        self.query_entry.insert(0, "1.1")

        btn_frame = ttk.Frame(top)
        btn_frame.grid(row=4, column=0, columnspan=6, sticky="w", pady=(12, 0))

        ttk.Button(btn_frame, text="Обчислити інтерполяції",
command=self.calculate).pack(side=tk.LEFT, padx=5)
        ttk.Button(btn_frame, text="Побудувати графіки",
command=self.show_graphs).pack(side=tk.LEFT, padx=5)

        self.info_var = tk.StringVar()
        self.info_var.set("Після обчислення тут з'являться значення точної функції
та результати інтерполяції.")
```

```
        ttk.Label(self, textvariable=self.info_var, wraplength=1000,
justify="left").pack(fill=tk.X, padx=10, pady=(5, 0))

        # таб вузлів
        nodes_box = ttk.LabelFrame(self, text="Вузли інтерполяції", padding=10)
        nodes_box.pack(fill=tk.X, padx=10, pady=10)

        self.nodes_tree = ttk.Treeview(nodes_box, columns=("x", "y"),
show="headings", height=6)
        self.nodes_tree.heading("x", text="x")
        self.nodes_tree.heading("y", text="y = 4 - cos(x^2)")
        self.nodes_tree.column("x", width=180, anchor="center")
        self.nodes_tree.column("y", width=220, anchor="center")
        self.nodes_tree.pack(side=tk.LEFT, fill=tk.X, expand=True)

        nodes_scroll = ttk.Scrollbar(nodes_box, orient="vertical",
command=self.nodes_tree.yview)
        self.nodes_tree.configure(yscrollcommand=nodes_scroll.set)
        nodes_scroll.pack(side=tk.RIGHT, fill=tk.Y)

        # таб результатів
        res_box = ttk.LabelFrame(self, text="Результати інтерполювання та відносна
похибка", padding=10)
        res_box.pack(fill=tk.BOTH, expand=True, padx=10, pady=(0, 10))

        self.result_tree = ttk.Treeview(
            res_box,
            columns=("method", "value", "error"),
            show="headings",
            height=6
        )
        self.result_tree.heading("method", text="Метод")
        self.result_tree.heading("value", text="y(x*)")
        self.result_tree.heading("error", text="Відносна похибка, %")
        self.result_tree.column("method", width=250, anchor="center")
        self.result_tree.column("value", width=250, anchor="center")
        self.result_tree.column("error", width=200, anchor="center")
        self.result_tree.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)

        res_scroll = ttk.Scrollbar(res_box, orient="vertical",
command=self.result_tree.yview)
        self.result_tree.configure(yscrollcommand=res_scroll.set)
        res_scroll.pack(side=tk.RIGHT, fill=tk.Y)
```

```
    # збирання даних
    def read_inputs(self):
        try:
            xs = [float(e.get().replace(",", ".")) for e in self.x_entries]
            xq = float(self.query_entry.get().replace(",", "."))
        except ValueError:
            raise ValueError("Усі значення мають бути числами. Використовуйте
крапку для десяткової частини.")

        if len(set(xs)) != len(xs):
            raise ValueError("Вузли x не повинні повторюватися.")

        xs_sorted = sorted(xs)

        if xq <= xs_sorted[0] or xq >= xs_sorted[-1]:
            raise ValueError("Точка x* має лежати всередині інтерполяційного
інтервалу.")

        for xi in xs_sorted:
            if abs(xq - xi) < 1e-12:
                raise ValueError("Точка x* не повинна збігатися з вузлом
інтерполяції.")

        return xs_sorted, xq

    # підрахунки
    def calculate(self):
        try:
            xs, xq = self.read_inputs()
            ys = [f_exact(x) for x in xs]
            y_exact = f_exact(xq)
            y_linear = local_linear_value(xs, ys, xq)
            y_quadratic = local_quadratic_value(xs, ys, xq)
            y_global = lagrange_value(xs, ys, xq)
            err_linear = relative_error(y_exact, y_linear)
            err_quadratic = relative_error(y_exact, y_quadratic)
            err_global = relative_error(y_exact, y_global)

            self.x_nodes = xs
            self.y_nodes = ys
            self.x_query = xq
```

```
            for item in self.nodes_tree.get_children():
                self.nodes_tree.delete(item)
            for x, y in zip(xs, ys):
                self.nodes_tree.insert("", tk.END, values=(f"{x:.6f}", f"{y:.6f}"))

            for item in self.result_tree.get_children():
                self.result_tree.delete(item)

            self.result_tree.insert("", tk.END, values=("Локальна лінійна інтерполяція",
f"{y_linear:.10f}", f"{err_linear:.6f}"))
            self.result_tree.insert("", tk.END, values=("Локальна квадратична
інтерполяція", f"{y_quadratic:.10f}", f"{err_quadratic:.6f}"))
            self.result_tree.insert("", tk.END, values=("Глобальна інтерполяція
Лагранжа", f"{y_global:.10f}", f"{err_global:.6f}"))

            # дані для графіків
            self.results = {
                "y_exact": y_exact,
                "y_linear": y_linear,
                "y_quadratic": y_quadratic,
                "y_global": y_global,
                "err_linear": err_linear,
                "err_quadratic": err_quadratic,
                "err_global": err_global,
            }
            self.info_var.set(
                f"Точка x* = {xq:.6f}\n"
                f"Точне значення y(x*) = {y_exact:.10f}\n"
                f"Лінійна: {y_linear:.10f}\n"
                f"Квадратична: {y_quadratic:.10f}\n"
                f"Глобальна: {y_global:.10f}"
            )

        except Exception as e:
            messagebox.showerror("Помилка", str(e))

    # створення графіків
    def show_graphs(self):
        if not self.x_nodes or not self.y_nodes or self.x_query is None:
            try:
                self.calculate()
            except Exception:
                return
```

```
        xs = self.x_nodes
        ys = self.y_nodes
        xq = self.x_query

        x_grid = np.linspace(xs[0], xs[-1], 700)
        exact_grid = np.array([f_exact(x) for x in x_grid], dtype=float)
        linear_grid = piecewise_linear_values(xs, ys, x_grid)
        quadratic_grid = piecewise_quadratic_values(xs, ys, x_grid)
        global_grid = np.array([lagrange_value(xs, ys, x) for x in x_grid], dtype=float)
        win = tk.Toplevel(self)
        win.title("Графіки інтерполювання")
        win.geometry("1200x1100")
        fig, axes = plt.subplots(3, 1, figsize=(11, 12), sharex=True)

        # 1 Локальна лінійна
        axes[0].plot(x_grid, exact_grid, label="Точна функція", color="black",
linewidth=2)
        axes[0].plot(x_grid, linear_grid, label="Локальна лінійна інтерполяція",
color="royalblue", linewidth=2)
        axes[0].scatter(xs, ys, color="orange", s=35, zorder=3, label="Вузли")
        axes[0].scatter([xq], [f_exact(xq)], color="red", s=60, marker="*", zorder=4,
label="x*")
        axes[0].set_title("Точна функція та локальна лінійна інтерполяція")
        axes[0].grid(True, linestyle=":")
        axes[0].legend()
        # 2 Локальна квадратична
        axes[1].plot(x_grid, exact_grid, label="Точна функція", color="black",
linewidth=2)
        axes[1].plot(x_grid, quadratic_grid, label="Локальна квадратична
інтерполяція", color="seagreen", linewidth=2)
        axes[1].scatter(xs, ys, color="orange", s=35, zorder=3, label="Вузли")
        axes[1].scatter([xq], [f_exact(xq)], color="red", s=60, marker="*", zorder=4,
label="x*")
        axes[1].set_title("Точна функція та локальна квадратична інтерполяція")
        axes[1].grid(True, linestyle=":")
        axes[1].legend()
        # 3 Глобальна інтерполяція
        axes[2].plot(x_grid, exact_grid, label="Точна функція", color="black",
linewidth=2)
        axes[2].plot(x_grid, global_grid, label="Глобальна інтерполяція Лагранжа",
color="darkorange", linewidth=2)
        axes[2].scatter(xs, ys, color="orange", s=35, zorder=3, label="Вузли")
```

```
        axes[2].scatter([xq], [f_exact(xq)], color="red", s=60, marker="*", zorder=4,
label="x*")
        axes[2].set_title("Точна функція та глобальна інтерполяція")
        axes[2].grid(True, linestyle=":")
        axes[2].legend()

        axes[2].set_xlabel("x")
        fig.suptitle("Порівняння точної функції та трьох видів інтерполяції",
fontsize=14, y=0.995)
        fig.tight_layout(rect=[0, 0, 1, 0.98])
        canvas = FigureCanvasTkAgg(fig, master=win)
        canvas.draw()
        canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)
        toolbar = NavigationToolbar2Tk(canvas, win)
        toolbar.update()
        canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)

if __name__ == "__main__":
    app = InterpolationApp()
    app.mainloop()
```

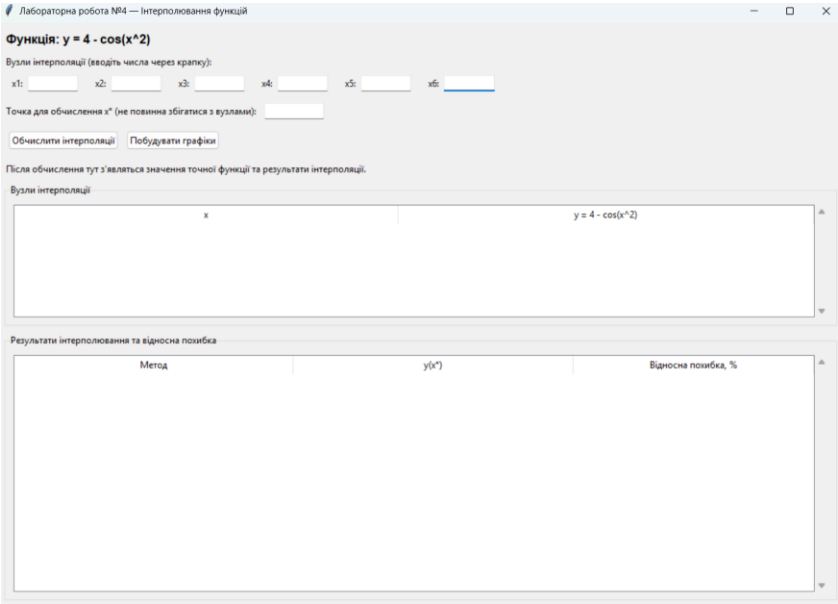
КІНЕЦЬ КОДУ

6. Результат роботи програми, графіки функції та інтерполянтів.

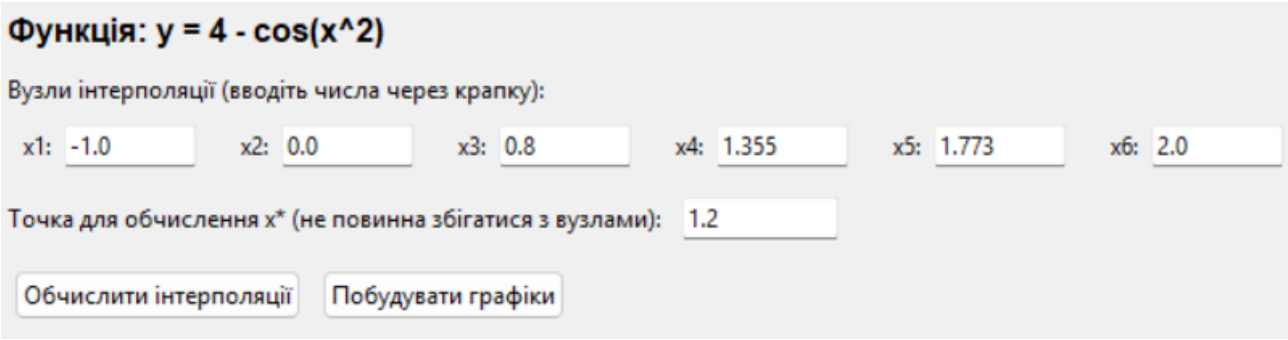
Дані: вузли інтерполяції **-1; 0; 0,8; 1,355; 1,773; 2;** довільне число **1,2**

Рівняння $y = 4-\cos(x^2)$

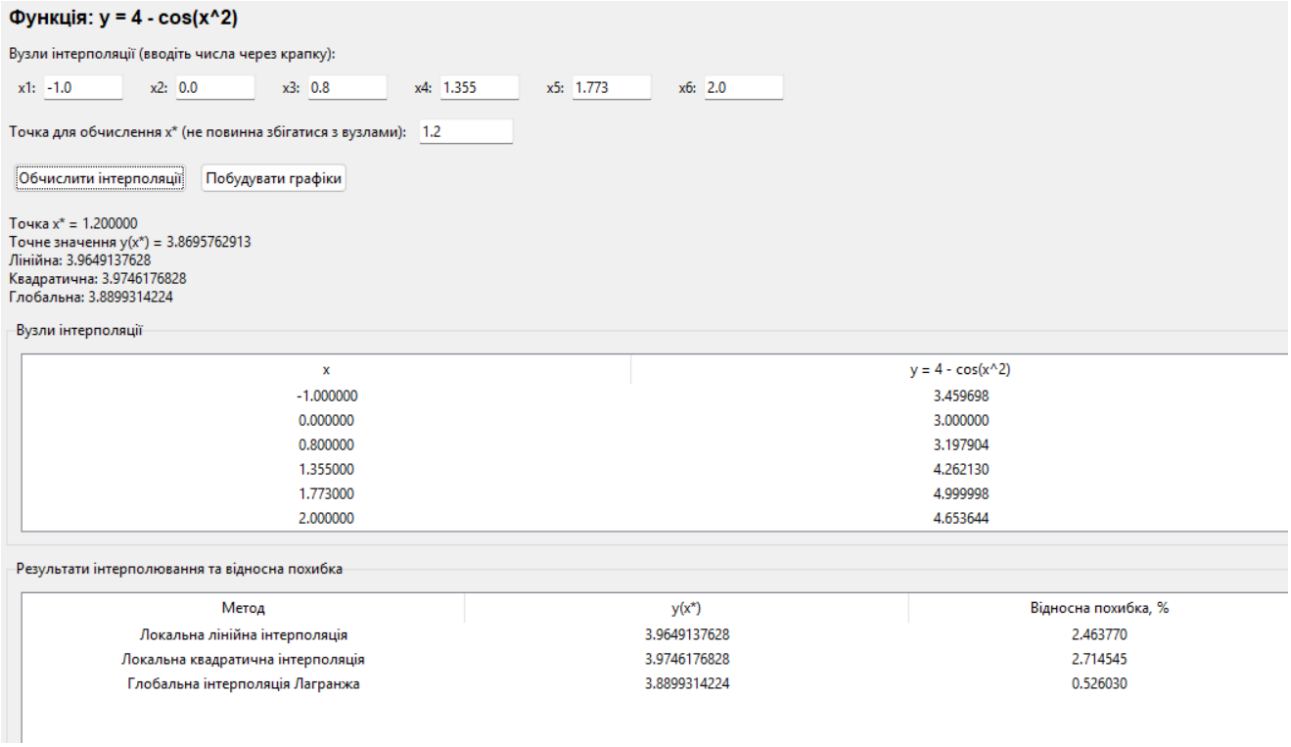
Запуск програми:



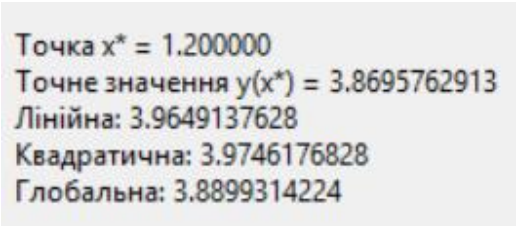
Введення вузлів інтерполяції та довільного числа:



Натискання на кнопку “Обчислити інтерполяції”:



Перегляд результату – підрахунки довільної точки:



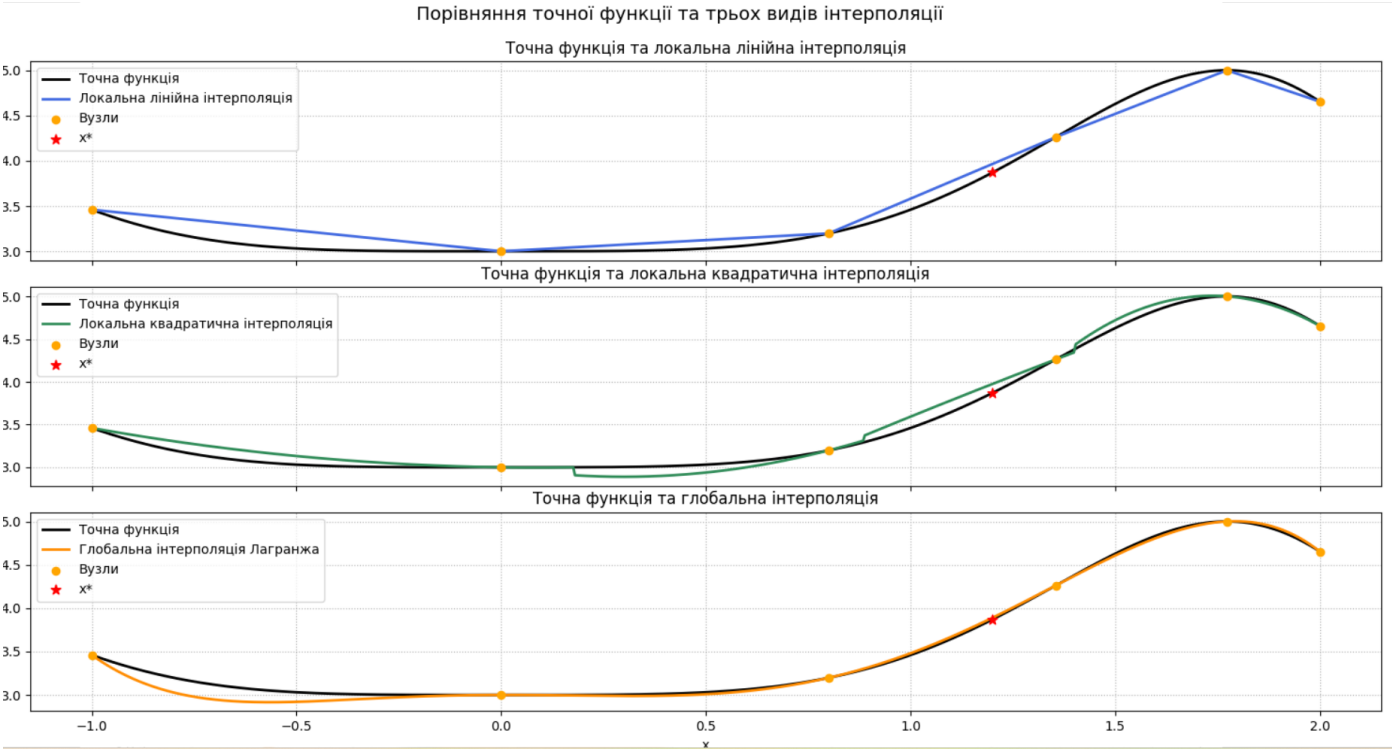
Перегляд результату – вузли інтерполяції:

Перегляд результату – підрахування довільної похибки:

Результати інтерполювання та відносна похибка

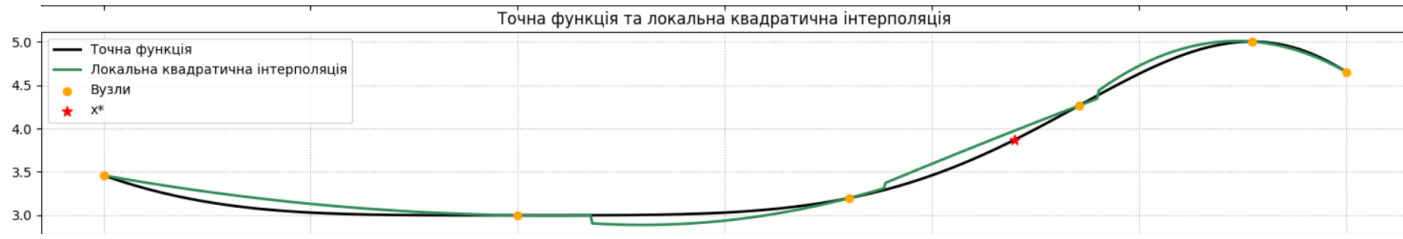
Метод	y(x*)	Відносна похибка, %
Локальна лінійна інтерполяція	3.9649137628	2.463770
Локальна квадратична інтерполяція	3.9746176828	2.714545
Глобальна інтерполяція Лагранжа	3.8899314224	0.526030

Натискання на кнопку “Побудувати графіки”:

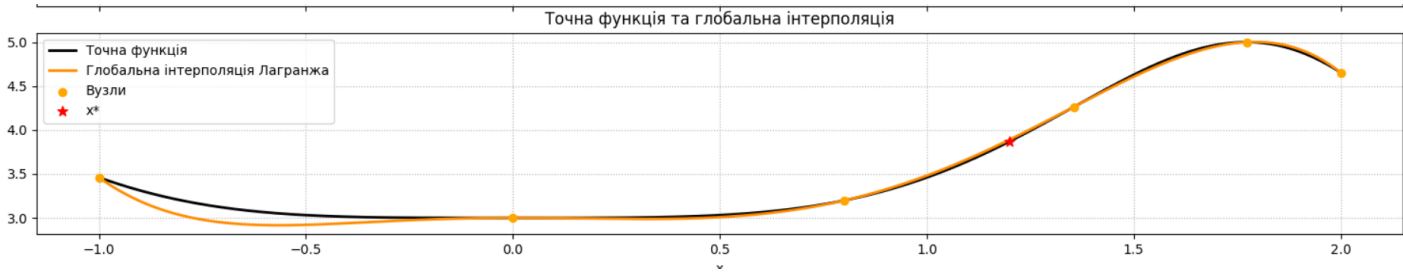


Перегляд результату – локальна лінійна інтерполяція:

Перегляд результату – локальна квадратична інтерполяція:



Перегляд результату – глобальна інтерполяція:



7. Висновки.

У ході виконання лабораторної роботи було проведено порівняльний аналіз трьох методів інтерполяції: локальної лінійної, локальної квадратичної та глобальної інтерполяції Лагранжа. Для оцінки точності обчислень було обрано довільну точку x* = 1,2, яка не збігається з жодним із вузлів.

Згідно з отриманими результатами, методи продемонстрували наступні показники відносної похибки:

- Глобальна інтерполяція Лагранжа: 0,526%;
- Локальна лінійна інтерполяція: 2,464%;
- Локальна квадратична інтерполяція: 2,715%.

Найбільш точним виявився метод глобальної інтерполяції Лагранжа. Це пояснюється тим, що даний метод будує поліном високого (п'ятого) ступеня, враховуючи інформацію з усіх шести вузлів одночасно. Це дозволяє максимально точно відтворити складну кривизну функції $y = 4 - \cos(x^2)$ на всьому інтервалі. Похибка у 0,53% є відмінним результатом для чисельних методів. Локальні методи показали значно більшу похибку (понад 2%).

Для функцій з нелінійним характером зміни, подібних до досліджуваної, найбільш ефективною є глобальна інтерполяція. Вона забезпечує мінімальну похибку за рахунок використання всього масиву даних вузлів. Локальні методи, хоч і є простішими в обчислювальній реалізації, доцільно використовувати лише на ділянках з малою зміною значень або при дуже високій щільності вузлів, де похибкою лінійності можна знехтувати.